

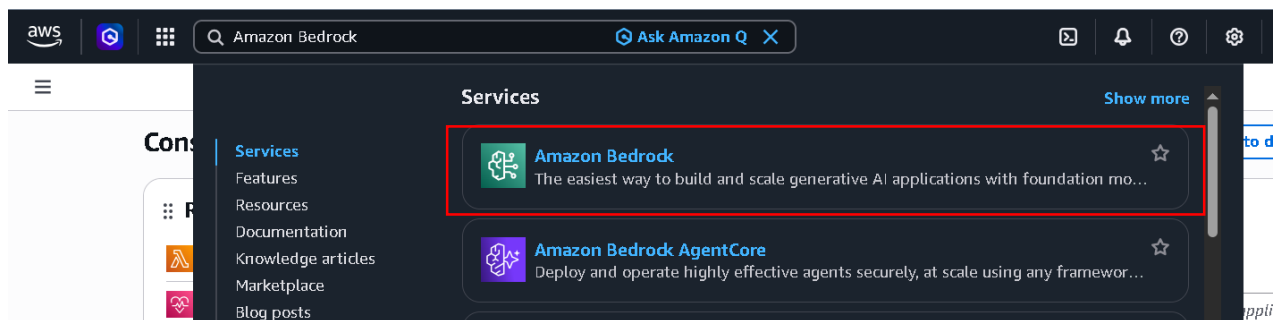
Web Scraper Agent

Summary: -

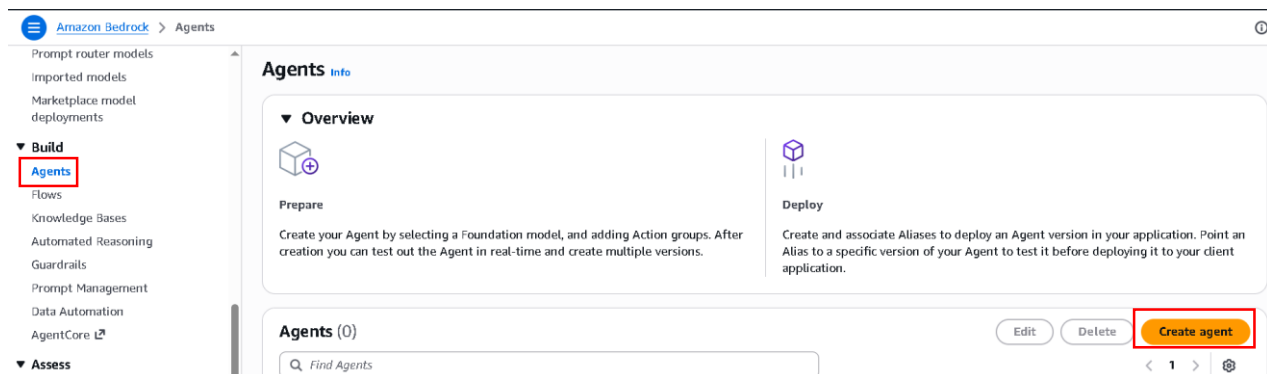
- In this project, I created a Web Scraper Agent using Amazon Bedrock that can crawl websites and extract useful information based on user queries. I configured the agent with proper instructions to act as a research analyst, capable of performing web searches and scraping content.
- I integrated an Action Group with a Lambda function using an OpenAPI schema to handle web scraping requests. The Lambda function was implemented using Python and libraries like BeautifulSoup to fetch, clean, and process website data.
- I also resolved deployment issues by adding required dependencies through a Lambda Layer and debugging errors using CloudWatch logs. Finally, I deployed, tested, and created an alias for the agent to ensure smooth execution.
- This system successfully retrieves web content and returns structured as well as human-readable responses.

Web Scraper Agent Creation: -

- Go to your Amazon Console and Search Amazon Bedrock



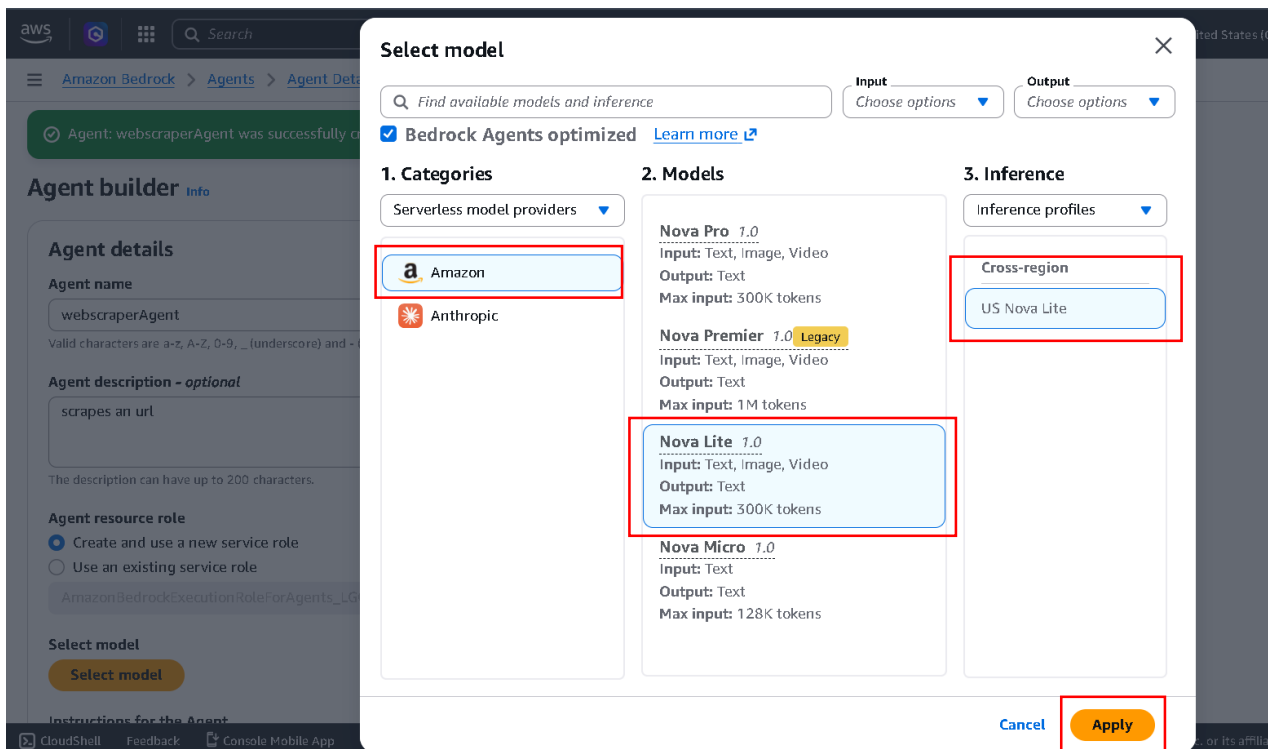
- Just click Amazon Bedrock.



- Scroll down in Navigation Bar and click Agents under Build.
- Now just click Create Agent.

- Give your Agent name
- And just click create.

- Our Agent is created.
- Now click Select Model.



- Select Model provider, Model, Inference.
- And just click Apply.

Select model

Nova Lite 1.0 ⓘ
US Nova Lite

Instructions for the Agent

Provide clear and specific instructions for the task the Agent will perform. You can also provide certain style and tone.

You are a research analyst that webscrapes the internet when provided a {question}. You use web searches to find relevant websites and information, or a web scraper to retrieve the content of individual webpages for review. Do not use both options unless explicitly told to do so. You should prefer information from reliable sources, such as the official website, Crunchbase, or news organizations. Some companies are startups and may be hard to find, so you should try multiple searches. Some websites will block the web scraper; you should try alternative sources. If you can't determine a reliable response based on the request provided, answer false. Your output should be a JSON document that includes the company name, a yes/no answer, and a summary of your explanation. If your output is an error, you should also respond with a JSON document that includes the error. Once you get the response, make sure to take that response and create a human-readable response (text).

This instruction must have a minimum of 40 characters.

► Additional settings

- Give Instruction for the agent like
 - You are a research analyst that webscrapes the internet when provided a {question}. You use web searches to find relevant websites and information, or a web scraper to retrieve the content of individual webpages for review. Do not use both options unless explicitly told to do so. You should prefer information from reliable sources, such as the official website, Crunchbase, or news organizations. Some companies are startups and may be hard to find, so you should try multiple searches. Some websites will block the web scraper; you should try alternative sources. If you can't determine a reliable response based on the request provided, answer false. Your output should be a JSON document that includes the company name, a yes/no answer, and a summary of your explanation. If your output is an error, you should also respond with a JSON document that includes the error. Once you get the response, make sure to take that response and create a human-readable response (text).
- Now Scroll Down.

Add

- ✔ Agent: webscraperAgent was successfully created.

Specify a Lambda or API Gateway and define a schema to specify the APIs that the agent can invoke to carry out its tasks.

- Define with API schemas

Agent responses in the test window will prompt the user for function details to generate a response. No further configurations are necessary.

- Quick create a new Lambda function

Action group schema

Select an existing schema or create a new one via the in-line editor to define the APIs that the agent can invoke to carry out its tasks.

☐ Select an existing API schema
Select from an existing S3 or define a schema

☒ Define via in-line schema editor
Use provided sample code or import and edit from S3

In-line OpenAPI schema

Import schema

Reset ↺

JSON ▼

```
1 {  
2   "openapi": "3.0.0",  
3   "info": {  
4     "title": "Insurance Claims Automation API",  
5     "version": "1.0.0",  
6     "description": "APIs for managing insurance claims by pulling a list of open claims, identifying outstanding
```

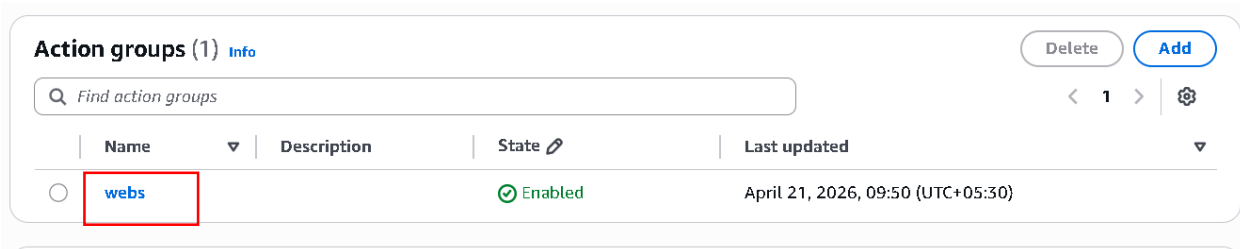
- Action group schema
 - Define via in-line schema editor
- Convert in-line OpenAPI schema into json
- And remove previous written JSON by this JSON code

```
{  
  "openapi": "3.0.0",  
  "info": {  
    "title": "Webscrape API",  
    "version": "1.0.0",  
    "description": "An API that will take in a URL, then scrape the internet to return the  
results."  
  },  
  "paths": {  
    "/search": {  
      "post": {  
        "summary": "Scrape content from the provided URL",  
        "description": "Takes in a URL and scrapes content from it.",  
        "operationId": "scrapeContent",  
        "requestBody": {  
          "required": true,  
          "content": {  
            "application/json": {  
              "schema": {  
                "type": "object",  
                "properties": {  
                  "inputURL": {  
                    "type": "string",  
                    "description": "The URL from which to scrape content"  
                  }  
                }  
              }  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

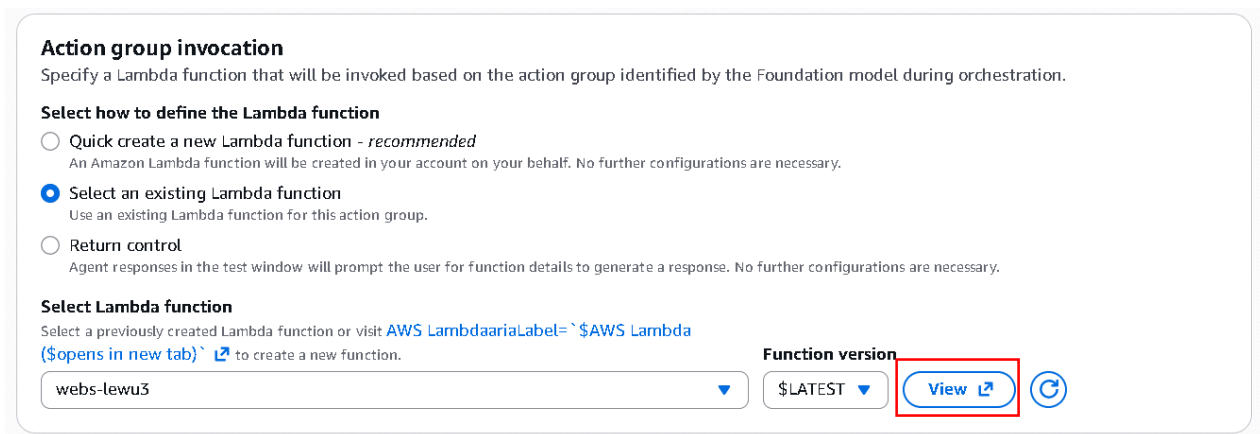
```
        },
        "required": [
            "inputURL"
        ]
    }
}
},
"responses": {
    "200": {
        "description": "Successfully scraped content from the URL",
        "content": {
            "application/json": {
                "schema": {
                    "type": "object",
                    "properties": {
                        "scraped_content": {
                            "type": "string",
                            "description": "The content scraped from the URL."
                        }
                    }
                }
            }
        }
    },
    "400": {
        "description": "Bad request. The input URL is missing or invalid."
    }
}
}
```



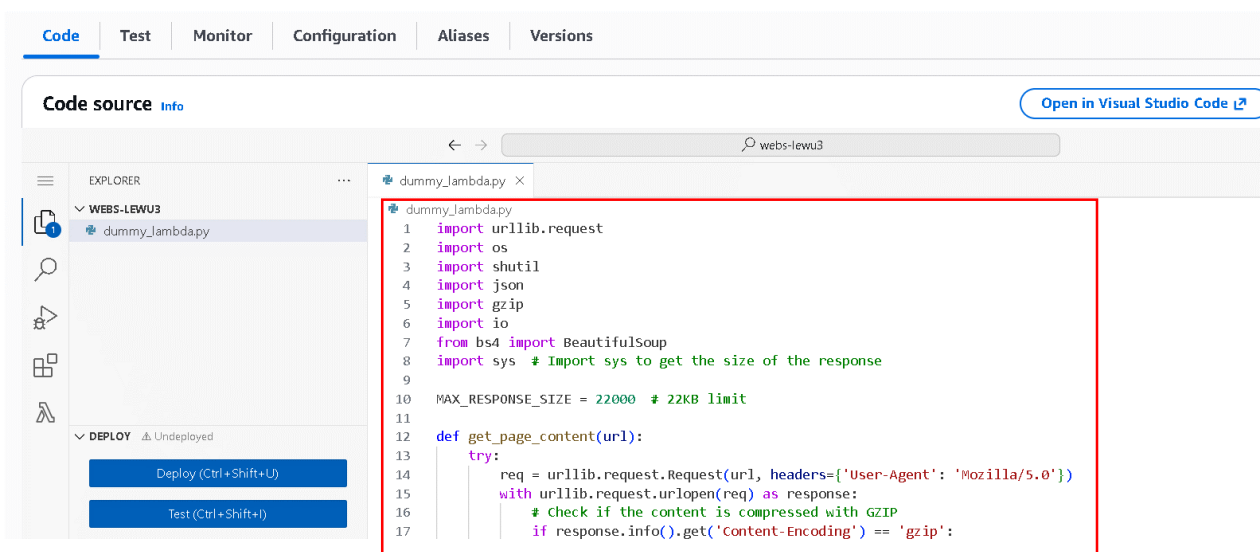
- Simply scroll down and click create.



- Now scroll down and see our webs action group is created.
- Let's click on it and go inside.



- Inside scroll down and click to View to see created Lambda function



- Now we see already written Lambda function
- We change with our code.

```

import urllib.request
import os
import shutil
import json
import gzip
import io

from bs4 import BeautifulSoup

import sys # Import sys to get the size of the response

MAX_RESPONSE_SIZE = 22000 # 22KB limit

def get_page_content(url):
    try:
        req = urllib.request.Request(url, headers={'User-Agent':
'Mozilla/5.0'})

        with urllib.request.urlopen(req) as response:
            # Check if the content is compressed with GZIP
            if response.info().get('Content-Encoding') == 'gzip':
                print(f"Content from {url} is GZIP encoded,
decompressing...")
                buf = io.BytesIO(response.read())
                with gzip.GzipFile(fileobj=buf) as f:
                    content = f.read().decode('utf-8')
            else:
                content = response.read().decode('utf-8')

            if response.geturl() != url: # Check if there were any
redirects

                print(f"Redirect detected for {url}")
                return None

            return content
    except Exception as e:
        print(f"Error while fetching content from {url}: {e}")
        return None

```



```

def empty_tmp_folder():
    try:
        for filename in os.listdir('/tmp'):
            file_path = os.path.join('/tmp', filename)
            if os.path.isfile(file_path) or
os.path.islink(file_path):
                os.unlink(file_path)
            elif os.path.isdir(file_path):
                shutil.rmtree(file_path)
        print("Temporary folder emptied.")
        return "Temporary folder emptied."
    except Exception as e:
        print(f"Error while emptying /tmp folder: {e}")
        return None

def save_to_tmp(filename, content):
    try:
        if content is not None:
            print(content)
            with open(f'/tmp/{filename}', 'w') as file:
                file.write(content)
            print(f"Saved {filename} to /tmp")
            return f"Saved {filename} to /tmp"
        else:
            raise Exception("No content to save.")
    except Exception as e:
        print(f"Error while saving {filename} to /tmp: {e}")
        return None

def check_tmp_for_data(query):
    try:
        data = []
        for filename in os.listdir('/tmp'):

```

```

        if query in filename:
            with open(f'/tmp/{filename}', 'r') as file:
                data.append(file.read())
            print(f"Found {len(data)} file(s) in /tmp for query {query}")
            return data if data else None
    except Exception as e:
        print(f"Error while checking /tmp for query {query}: {e}")
        return None

def handle_search(event):
    # Extract inputURL from the requestBody content
    request_body = event.get('requestBody', {})
    input_url = ''

    # Check if the inputURL exists within the properties
    if 'content' in request_body:
        properties = request_body['content'].get('application/json',
        {}).get('properties', [])
        input_url = next((prop['value'] for prop in properties if
        prop['name'] == 'inputURL'), '')

    # Handle missing URL
    if not input_url:
        return {"error": "No URL provided"}

    # Ensure URL starts with http or https
    if not input_url.startswith(('http://', 'https://')):
        input_url = 'http://' + input_url

    # Check for existing data in /tmp
    tmp_data = check_tmp_for_data(input_url)
    if tmp_data:
        return {"results": tmp_data}

```

```

# Clear /tmp folder
empty_tmp_result = empty_tmp_folder()
if empty_tmp_result is None:
    return {"error": "Failed to empty /tmp folder"}

# Get the page content
content = get_page_content(input_url)
if content is None:
    return {"error": "Failed to retrieve content"}

# Parse and clean the HTML content
cleaned_content = parse_html_content(content)

# Save the content to /tmp
filename = input_url.split('/')[-1].replace('/', '_') + '.txt'
save_result = save_to_tmp(filename, cleaned_content)

if save_result is None:
    return {"error": "Failed to save to /tmp"}

# Check the size of the response and truncate if necessary
response_data = {'url': input_url, 'content': cleaned_content}
response_size = sys.getsizeof(json.dumps(response_data))

if response_size > MAX_RESPONSE_SIZE:
    print(f"Response size {response_size} exceeds limit.
Truncating content...")
    truncated_content = cleaned_content[: (MAX_RESPONSE_SIZE -
response_size)]
    response_data['content'] = truncated_content

    return {"results": response_data}

def parse_html_content(html_content):

```

```

soup = BeautifulSoup(html_content, 'html.parser')
for script_or_style in soup(["script", "style"]):
    script_or_style.decompose()
text = soup.get_text()
lines = (line.strip() for line in text.splitlines())
chunks = (phrase.strip() for line in lines for phrase in
line.split(" "))
cleaned_text = '\n'.join(chunk for chunk in chunks if chunk)

max_size = 25000
if len(cleaned_text) > max_size:
    cleaned_text = cleaned_text[:max_size]

return cleaned_text

def lambda_handler(event, context):
    response_code = 200
    action_group = event['actionGroup']
    api_path = event['apiPath']

    print("THE EVENT: ", event)

    if api_path == '/search':
        result = handle_search(event)
    else:
        response_code = 404
        result = f"Unrecognized api path: {action_group}::{api_path}"

    response_body = {
        'application/json': {
            'body': result
        }
    }

```

```

action_response = {
    'actionGroup': event['actionGroup'],
    'apiPath': event['apiPath'],
    'httpMethod': event['httpMethod'],
    'statusCode': response_code,
    'responseBody': response_body
}

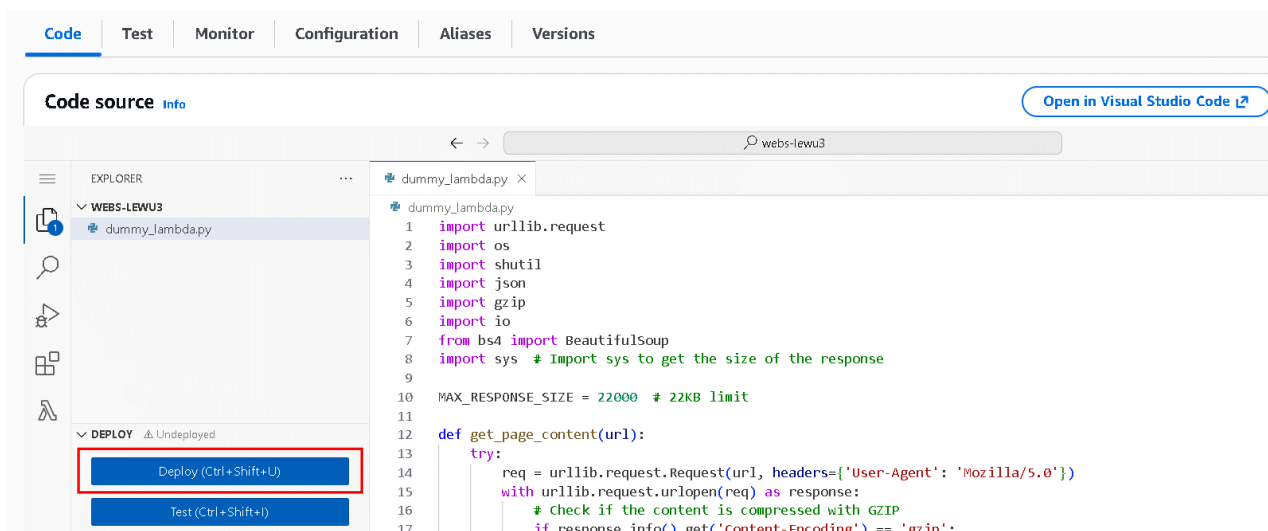
api_response = {'messageVersion': '1.0', 'response':
action_response}

print("action_response: ", action_response)
print("response_body: ", response_body)

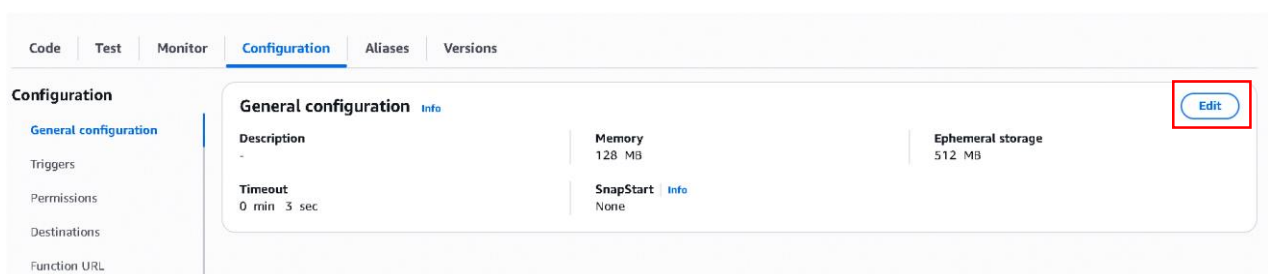
return api_response

```

- Now this code gives a result and then this result is going to be then passed back to AWS or Amazon Bedrock Agent to then take that information and create a coherent response.



- Now simply click Deploy



- Go to configuration and click Edit

Memory [Info](#)

Your function is allocated CPU proportional to the memory configured.

3000 MB

Set memory to between 128 MB and 10240 MB

Ephemeral storage [Info](#)

You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)

1024 MB

Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)

Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review .NET runtimes, [view pricing](#).

None

Supported runtimes: .NET 10 (C#/F#/PowerShell), .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.

Timeout

1 min 0 sec

- Now just scroll down and click save.
- Now lambda function work is done go back to your Agent console.

Action status

When enabled, your action group is influencing what the response of your Agent will be. Disable Action to stop it from impacting the Agent's responses.

- ☒ Enable
☐ Disable

Cancel

Save

Save and exit

- Just scroll down and click save and exit.

[Amazon Bedrock](#) > [Agents](#) > [Agent Details](#) > Agent builder

Agent builder [Info](#)

Manual

Assistant

Test

Prepare

Save

Save and exit

Agent details

Agent name

webscraperAgent

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Agent description - optional

scrapes an url

The description can have up to 200 characters.

- Now just click save
- After save just click prepare.
- And then test your agent.

Test Agent

Using ODT [Change](#)

tell me what this url about
https://en.wikipedia.org/wiki/Amazon_Web_Services

Your request couldn't be completed. Lambda function arn:aws:lambda:us-west-2:463646775279:function:webs-lewu3 encountered a problem while processing request. The error message from the Lambda function is Unhandled. Check the Lambda function log for error details, then try your request again after fixing the error.

[Show trace >](#)
Failed

[Trace](#)
[Session summaries](#)

Trace (0)

[<](#)
[Pre-Processing Trace](#)
[Routing Trace](#)

Pre-Processing Trace

After running an input in the test window, this pre-processing trace a input was identified as malicious or outside of the agent's domain.

- You see I ask but request failed
- So let's resolve this
- Go back to tour lambda tab

webs-lewu3

[Throttle](#)
[Copy ARN](#)
[Actions](#)

Function overview [Info](#)

[Diagram](#)
[Template](#)

webs-lewu3

Layers (0)

[+ Add trigger](#)
[+ Add destination](#)

[Code](#)
[Test](#)
[Monitor](#)
[Configuration](#)
[Aliases](#)
[Versions](#)

[Export to Infrastructure Composer](#)
[Download](#)

Function ARN

arn:aws:lambda:us-west-2:463646775279:function:webs-lewu3

Description

-

Last modified

13 minutes ago

[Code](#)
[Test](#)
[Monitor](#)
[Configuration](#)
[Aliases](#)
[Versions](#)

Filter metrics by

Function

[View CloudWatch logs](#)

- Now in your lambda function click Monitor.
- Then in monitor click view cloudWatch logs.

[Log streams](#)
[Tags](#)
[Data protection](#)
[Anomaly detection](#)
[Metric filters](#)
[Subscription filters](#)
[Contributor Insights](#)
[Field indexes](#)
[Transformer](#)

Log streams (1)

[Delete](#)
[Create log stream](#)
[Search all log streams](#)

☐ Exact match
☐ Show expired
[Info](#)

☐ Log stream

Last event time

☐
[2026/04/21/\[LATEST\]a8fa1b326abd413c87d9113a9b56ac88](#)

2026-04-21 04:50:18 (UTC)

- Now you are redirected to new tab cloudWatch.
- Now just scroll down and in log stream click the log that you see

Log events

[Actions](#)
[Start tailing](#)
[Create metric filter](#)

[Clear](#)
[1m](#)
[30m](#)
[1h](#)
[12h](#)
[Custom](#)

UTC timezone

Display

Timestamp

Message

No older events at this moment. [Retry](#)

2026-04-21T04:50:18.543Z

INIT_START Runtime Version: python:3.12.mainlinev2.v6 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:dbfa6aec8278478c1512458be8c7a99b2d63682d2e2d1e8d276dbf05b7f99755

2026-04-21T04:50:18.675Z

[WARNING] 2026-04-21T04:50:18.675Z LAMBDA WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update...

2026-04-21T04:50:18.675Z

[ERROR] Runtime.ImportModuleError: Unable to import module 'dummy_lambda': No module named 'bs4' Traceback (most recent call last):

[ERROR] Runtime.ImportModuleError: Unable to import module 'dummy_lambda': No module named 'bs4'

Traceback (most recent call last):

2026-04-21T04:50:18.691Z

INIT_REPORT Init Duration: 148.26 ms Phase: Init Status: error Error Type: Runtime.ImportModuleError

- Now you are in logs page.
- You see in third Line we get error.
- See that it says unable to import module 'dummyslambda'.
- Basically it means we are actually importing this beautiful soup IP and all of these other libraries.
- But what happened is that in this environment some of these libraries are not there.
- So which means they are not available.
- So what we can do we attach a layer in our lambda function.
- So go to your lambda function tab.

webs-lewu3

▼ **Function overview** [Info](#)

Diagram | **Template**

 **webs-lewu3**

 **Layers** (0)

[+ Add trigger](#) [+ Add destination](#)

Code | **Test** | **Monitor** | **Configuration** | **Aliases** | **Versions**

- Now click on Layers

Layers (0) [Info](#) Last fetched 21/04/2026, 10:20:53 [Edit](#)

Merge order	Name	Layer version	Compatible runtimes	Compatible architectures	Version ARN
There is no data to display.					

- Now simply scroll down and in Layer Section.
- Click Edit.

Layers [Info](#)

No layers are configured on the function.

[Add a layer](#)

- Now you see Add a layer.
- But before you click you need to create a Layer
- So go to lambda function in a new tab.

Lambda > **Layers** Last fetched 21/04/2026, 10:50:42 [Create layer](#)

Name	Version	Description	Compatible runtimes	Compatible architectures	Created
There is no data to display.					

Lambda

Dashboard

Applications

Functions

▼ **Function resources**

Capacity providers [New](#)

Code signing configurations

Event source mappings

Layers

Replicas

- Now just click Layer in Navigation bar under Functions section.
- And after this click create layer

Create layer

Layer configuration

Name

googlesearch_requests_layer

Description - optional

☒ Upload a .zip file
☐ Upload a file from Amazon S3

[Choose file](#)

layer-python-requests-googlesearch-beautifulsoup.zip
1.47 MB

For files larger than 10 MB, consider uploading using Amazon S3.

- Give name to your layer.
- Select upload a .zip file.
- Click choose file and upload the file. (that file I provided in resource)
- Now just scroll down

Compatible architectures - optional | [Info](#)
Choose the compatible instruction set architectures for your layer.

Search Compatible architectures

x86_64

Compatible runtimes - optional | [Info](#)
Choose up to 15 runtimes.

Search Compatible runtimes

Python 3.12

License - optional | [Info](#)

[Cancel](#) [Create](#)

- Now just click Create.

Lambda > Layers > googlesearch_requests_layer > 1

Successfully created layer googlesearch_requests_layer version 1.

googlesearch_requests_layer [Delete](#) [Download](#) [Create version](#)

Version details

Version 1	Version ARN arn:aws:lambda:us-west-2:463646775279:layer:googlesearch_requests_layer:1	Description -
Created 6 minutes ago	License -	Compatible runtimes python3.12
Compatible architectures x86_64		

- Now our layer is successfully created.
- Now go back to your Lambda function tab.

Layers [Info](#)

No layers are configured on the function.

[Add a layer](#)

- Now just click Add a layer

Choose a layer ✕

Layer source [Info](#)

Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ AWS layers
Choose a layer from a list of layers provided by AWS.
 ☒ Custom layers
Choose a layer from a list of layers created by your AWS account.
 ☐ Specify an ARN
Specify a layer by providing the ARN.

Custom layers

Layers created by your AWS account.

googlesearch_requests_layer ↕

Version

1 ↕

Description

-

Compatible runtimes

Python 3.12

Compatible architectures

x86_64

[Cancel](#) [Add](#)

- choose custom Layer.
- Select your created Layers.
- And choose version as 1.
- And now just click Add.

Layers [Info](#)

googlesearch_requests_layer ✕

arn:aws:lambda:us-west-2:463646775279:layer:googlesearch_requests_layer:1

[Add a layer](#)

Some Lambda layers may not be compatible with functions running on Lambda Managed instances. Verify layer compatibility and remove any incompatible layers from your function configuration. Layer compatibility depends on the runtime and architecture of your EC2 instance.

[Cancel](#) [Save](#)

- Now just click Save.
- Go now go to your Amazon Bedrock Agent tab.

Agent builder [Info](#)

[Manual](#) | [Assistant](#) [Test](#) [Prepare](#) [Save](#) [Save and exit](#)

Agent details

Agent name

webscraperAgent

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Agent description - optional

scrapes an url

The description can have up to 200 characters.

- Now click Save.
- After Save Prepare.
- After Prepare Test.

Test Agent

Using ODT [Change](#)

tell me about this link: <https://docs.crawl4ai.com/>

[{"company": "Crawl4AI", "answer": "yes", "summary": "Crawl4AI is an open-source web crawler and scraper designed to be LLM-friendly. It offers features such as generating clean markdown, structured extraction, advanced browser control, high performance, and is completely open source. The documentation includes sections on setup and installation, quick start guides, core functionalities, advanced features, extraction strategies, and API references. It also provides code samples and encourages community contributions. The project aims to democratize data access and empower users with speed, cost-efficiency, and creative freedom in data extraction and AI workflows."}]

< Hide trace

Enter your message here

:

Trace

Session summaries

Trace (2)

Request ID d9552571-9bf7-4139-9b37-4a725221af1d

< ace

Routing Trace

Orchestration and Knowledge Base

Orchestration and Knowledge Base

After running a prompt in the test window, this orchestration trace allows you to explore the tra
thought used by the agent's orchestration prompt component. If a knowledge base was invoked,
results from the knowledge base were summarized to generate an observation that is used for fo
generating the response.

▶ Trace step 1

▶ Trace step 2

- I ask now it's working perfectly.
- Now ask some other Question.

now give me 4-5 main points that you find here:
<https://docs.crawl4ai.com/core/deep-crawling/>

Here are 4-5 main points from the provided link:
1. **Deep Crawling Strategy Options:** Crawl4AI offers three main deep crawling strategies: BFSDeepCrawlStrategy (breadth-first search), DFSDeepCrawlStrategy (depth-first search), and BestFirstCrawlingStrategy (recommended for intelligent crawling). Each strategy has parameters like max_depth, include_external, max_pages, and score_threshold.
2. **Streaming vs. Non-Streaming Results:** Crawl4AI can return results in two modes: non-streaming (default) and streaming. Non-streaming mode waits for all results to be collected before returning, while streaming mode

Enter your message here

:

Run

Trace

Session summaries

Trace (2)

Request ID d9552571-9bf7-4139-9b37-4a725221af1d

< ace

Routing Trace

Orchestration and Knowledge Base

Orchestration and Knowledge Base

After running a prompt in the test window, this orchestration trace allows you to explore the t
thought used by the agent's orchestration prompt component. If a knowledge base was invoke
results from the knowledge base were summarized to generate an observation that is used for
generating the response.

▶ Trace step 1

▶ Trace step 2

- It's working Perfectly.

Amazon Bedrock > Agents > Agent Details > Agent builder

Agent builder [Info](#)

Manual | Assistant | Test | Prepare | Save | **Save and exit**

Agent details

Agent name

webscraperAgent

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Agent description - optional

scrapes an url

The description can have up to 200 characters.

- For one more time click save, then Prepare.
- Now just click Save and Exit.

Amazon Bedrock > Agents > Agent Details

webscraperAgent

Create Alias Test Edit in Agent Builder

Agent overview

Name
webscraperAgent

Description
scrapes an url

Creation date
April 21, 2026, 09:33 (UTC+05:30)

Permissions
arn:aws:iam::463646775279:role/service-role/AmazonBedrockExecutionRoleForAgents_OHMF2RRYG

User Input
DISABLED

Idle session timeout
600 seconds

ID
PMIZODPKR5

Status
✔ PREPARED

Last prepared
April 21, 2026, 11:41 (UTC+05:30)

Agent ARN
arn:aws:bedrock:us-west-2:463646775279:agent/PMIZODPKR5

Memory
Disabled

KMS key
-

- Now simply click on create Alias.

Amazon Bedrock > Agents > Agent Details

webscraperAgent

Agent overview

Name
webscraperAgent

Description
scrapes an url

Creation date
April 21, 2026, 09:33 (UTC+05:30)

Permissions
arn:aws:iam::463646775279:role/service-role/AmazonBedrockExecutionRoleForAgents_OHMF2RRYG

User Input
DISABLED

Idle session timeout
600 seconds

Tags (0)

Create alias ✕

Alias name
webscraper-alias-final
Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Description - optional

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 200 characters.

Associate a version
☒ Create a new version and associate it to this alias.
☐ Use an existing version to associate this alias.

Select throughput
☒ On-demand (ODT)
☐ Provisioned Throughput (PT)

Cancel Create alias

- Now simply write alias name.
- Associate a version
 - Create a new version and associate it to this alias.
- Select throughput
 - On-demand
- Now just simply click create alias.

✓ Alias: webscraper-alias-final was successfully created.

webscraperAgent

Create Alias

Test

Edit in Agent Builder

Agent overview

Name

webscraperAgent

ID

PMIZODPKR5

Description

scrapes an url

Status

✓ PREPARED

Creation date

April 21, 2026, 09:33 (UTC+05:30)

Last prepared

April 21, 2026, 11:41 (UTC+05:30)

- Our Alias is created now just click Test.



✓ Alias: webscraper-alias-final was successfully created.

Test

Using ODT [Change](#)

TestAlias: Working draft [Info](#)

TestAlias: Working draft

webscraper-alias-final: Version 1

Trace

Session summaries

- Now select your created alias
- Now check it's running or not

Test

Using ODT [Change](#)

webscraper-alias-final: Version 1 [Info](#)



tell me about this url: <https://docs.crawl4ai.com/core/deep-crawling/>



{"company_name": "Crawl4AI", "answer": "true", "summary": "The URL provided is from the Crawl4AI documentation, which explains the deep crawling feature of the Crawl4AI tool. The documentation covers various aspects of deep crawling, including different strategies, filtering techniques, and advanced configurations. It provides examples and explanations for setting up and customizing deep crawls to extract specific content from <redacted>ites."}



< Hide trace

Trace

Session summaries

Trace (2)

Request ID eb7ea66f-ef88-424c-849b-ac7bdaa

< ace

Routing Trace

Orchestra

Orchestration and Knowledge Base

After running a prompt in the test window, this orchestration prompt was used by the agent's orchestration prompt component. The results from the knowledge base were summarized to generate the response.

► Trace step 1

- It's working Properly.